databricks-genre-model, explained step by step

From the Chinook view to a scheduled Databricks job and a token-protected serving endpoint

This document walks through everything that was built for the second part of the *Bases de Datos* assignment: the same music-genre model as `pipeline-db-ml`, now trained every day by a Databricks job and published on a Databricks Model Serving endpoint that anyone with the URL and an access token can query.

Each step explains **what** was done, **why** it was done that way and **how** to check that it works, with the real code, the real commands and the real output of the first run.

Repository: https://github.com/consvzgz-up/databricks-modelo-genero
Endpoint URL: https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-cliente/invocations
Date: October 9, 2026

Contents

1. The big picture
2. Vocabulary you need
3. Step 0. What we kept from pipeline-db-ml
4. Step 1. The data: from the Postgres dump to CSVs to Delta tables
5. Step 2. The bundle and the Databricks CLI
6. Step 3. Training and registering the model
7. Step 4. The serving endpoint
8. Step 5. The smoke test and querying from outside
9. Step 6. The schedule and what each run does
10. Step 7. The token and the delivery to the professor
11. Operating it: runs, costs, scale to zero, tearing down
12. Appendix. Full source of the four task files and the wrapper

1. The big picture

The question is the same as in the first assignment: a customer walks into the music store, we know three things about them (what kind of email they use, their country and their city) and we want to guess which music genre they are most likely to buy. What the assignment asks for now is different infrastructure:

1. A **Databricks job** that trains the model **periodically**.
2. The trained model **published on a Databricks serving endpoint**.
3. For the professor: the **endpoint URL**, an **access token** and an **example of how to query it**.

Everything lives in one Git repository. One command (`databricks bundle deploy`) copies the code and the data into the Databricks workspace and creates the job; the job then does the rest by itself, every day.

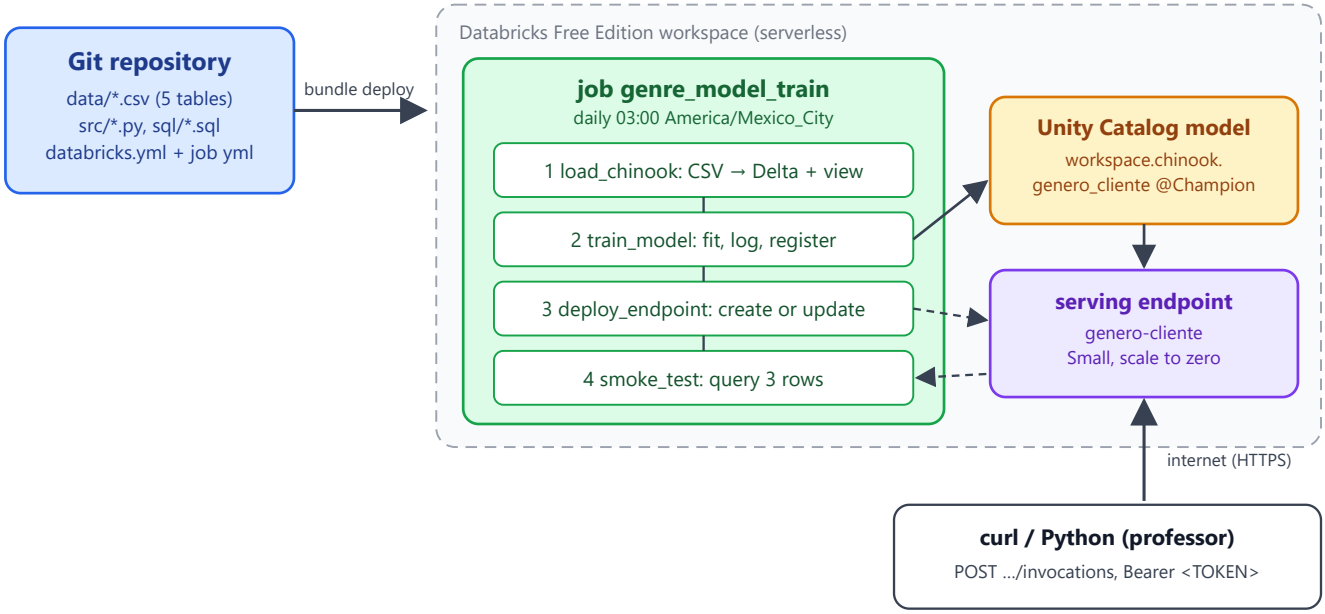


Figure 1. The flow. The repository is deployed once; the job then loads, trains, registers, publishes and tests on every run. Dashed arrows: task 3 changes the endpoint, task 4 queries it.

Compare with the first assignment. There, a cron container wrote a `.pkl` file into a shared folder and a FastAPI container read it. Here the "file" is a **versioned model in a registry**, the "cron" is a **scheduled job**, and the "API container" is a **managed endpoint**. Nobody administers a virtual machine.

2. Vocabulary you need

Databricks Free Edition. A free Databricks account with one workspace. It runs only *serverless* compute, allows at most 5 concurrent job tasks per account, a limited number of serving endpoints (CPU only), outbound internet only to a list of trusted domains, and a daily fair-use quota: if you exceed it, compute is switched off for the rest of the day. (free-edition-limitations)

Serverless compute. You do not create or size a cluster. You submit work and Databricks provides the machines, then releases them. For a Python task you only declare which libraries it needs (an *environment*).

Asset bundle. A folder with a `databricks.yml` file that describes code, files and resources (jobs, schemas...) as YAML. `databricks bundle deploy` uploads the folder to the workspace and creates or updates the resources; `databricks bundle run` starts a job. The docs now also call them "Declarative Automation Bundles"; the commands are the same.

Job and task. A *job* is a named, schedulable unit of work. It contains *tasks* (here, four Python scripts) with dependencies between them: a task starts only when the ones it `depends_on` have succeeded. Each execution is a *run*, with its own id, log and result.

Delta table and Unity Catalog. A *Delta table* is Databricks' table format (Parquet files plus a transaction log). *Unity Catalog* is the governance layer that names everything with three levels, `catalog.schema.object` : our tables are `workspace.chinook.customer` and so on, the view is `workspace.chinook.vw_cliente_genero` and the model is `workspace.chinook.genero_cliente` .

MLflow run and registered model. An MLflow *run* is one training session with its parameters, metrics (our `accuracy`) and files (our classification report). A *registered model* is a named entry in the registry; every time we log a new model under that name it gets a new *version* (1, 2, 3...).

Alias. A movable label that points to one version of a registered model, like a Git tag you can move. We use `Champion` : "the version that should be served". Code loads `models:/workspace.chinook.genero_cliente@Champion` without knowing the number.

pyfunc model. MLflow's generic Python model: a class with a `predict(context, model_input)` method. Wrapping the scikit-learn pipeline in a pyfunc lets us decide the output format (genre plus top 3) and clean the input before predicting.

Serving endpoint. A managed HTTPS service that loads a registered model version and answers `POST /serving-endpoints/<name>/invocations` with predictions. It has a size (`Small`) and can be updated to a new version without downtime.

Scale to zero. An endpoint option: when there is no traffic the endpoint releases its compute and costs nothing; the next request wakes it up, so the first call after an idle period takes longer.

Personal access token (PAT). A long random string that acts as a password for the Databricks REST API on behalf of the user who created it. It is sent in the header `Authorization: Bearer <token>` . It has a lifetime, can be revoked, and must never be committed. (dev-tools/auth/pat)

Quartz cron. The schedule syntax Databricks jobs use. Unlike Linux cron it has a **seconds** field first: `0 0 3 * * ?` reads "second 0, minute 0, hour 3, every day of the month, every month, any day of the week", that is, every day at 03:00. The `?` means "no specific value" and is required in one of the two day fields.

3. Step 0. What we kept from pipeline-db-ml

The first assignment (`consvzgz-up/pipeline-db-ml` , delivered September 18, 2026) already had a working model, a database view and an API contract. The cheapest correct move was to keep all of that and change only the infrastructure around it.

Piece	pipeline-db-ml (EC2)	databricks-genre-model
The view	`vw_cliente_genero` in Postgres	same SQL, names `workspace.chinook.*`
Features, target	`tipo_correo, pais, ciudad` → `genero`	same
Pipeline	`OneHotEncoder(handle_unknown="ignore")` → `LogisticRegression(max_iter=1000)`	same
Measurement	20 % hold-out, `random_state=42` , no stratify	same
Email rule	`PredictorRequest.extraer_tipo_correo`	same function in `src/genre_model.py`
Answer	`genero` + `top_3` list	`genero` + `top_1..3` , `p_1..3`
Retraining	cron, every hour	scheduled job, every day at 03:00
Model storage	`assets/modelo_entrenado.pkl`	MLflow registry in Unity Catalog, alias `Champion`
Serving	FastAPI on port 8000, open to the internet	Model Serving endpoint, token required

Two things could **not** be kept:

- **The database connection.** On Free Edition, outbound internet is "restricted to a limited set of trusted domains", so a job cannot open a connection to the Postgres on EC2 or to Supabase. The data has to live inside Databricks (Step 1).
- **The nested `top_3` list.** A model in Unity Catalog needs a *signature* (a declared input and output schema). Flat columns (`top_1` , `p_1` , ...) give a simple, clean signature; the information is the same.

4. Step 1. The data: from the Postgres dump to CSVs to Delta tables



Figure 2. How the Chinook data gets into Databricks without any database connection.

4.1 Why CSVs, and why only five tables

What. `scripts/chinook_to_csv.py` reads the same dump the first assignment loaded into Postgres and writes the five tables the view needs: `genre`, `track`, `invoice_line`, `invoice` and `customer`. The CSVs are committed (they are small) and travel to the workspace with the bundle.

Why. The job cannot reach any outside database (Free Edition limitation). Parsing the dump needs no server and no download, and it guarantees the data is byte-for-byte the data of the first assignment.

The dump writes one row per line, with SQL Server style `N'...'` strings and bare `NULL`s:

`pipeline-db-ml/db/init/01_chinook.sql (lines 222 to 224 and 4405)`

```
INSERT INTO genre (genre_id, name) VALUES
    (1, N'Rock'),
    (2, N'Jazz'),
...
    (2, N'Leonie', N'Köhler', NULL, N'Theodor-Heuss-Straße 34', N'Stuttgart', NULL, N'Germany', N'70174',
N'+49 0711 2842222', NULL, N'leonekohler@surfeu.de', 5),
```

The script drops the `N` prefix and marks `NULL` only *outside* quoted strings, then lets Python's `csv` module (with `'` as the quote character) split the values. The full parser is in the repository; its self-test checks the counts and a few tricky values:

How to check.

```
$ python scripts/chinook_to_csv.py --self-test
genre            25  ok
track          3503  ok
invoice_line   2240  ok
invoice         412  ok
customer         59  ok
customer 1 email: ok
customer 1 first_name: ok
customer 2 company is NULL: ok
track 1 name: ok
track 2001 name (escaped quote): ok
N and NULL inside strings untouched: ok
csv round trip: ok
self-test passed
```

4.2 Task 1: CSV to Delta tables, then the view

src/load_chinook.py (main)

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", required=True)
    args = ap.parse_args()

    spark = SparkSession.builder.getOrCreate()
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SCHEMA}")

    for table in TABLES:
        path = f"{args.data_dir.rstrip('/')}/{table}.csv"
        raw, how = read_csv(spark, path)
        df = typed(table, raw)
        (df.write.format("delta").mode("overwrite").option("overwriteSchema", "true")
           .saveAsTable(f"{CATALOG}.{SCHEMA}.{table}"))
        n = spark.table(f"{CATALOG}.{SCHEMA}.{table}").count()
        print(f"{CATALOG}.{SCHEMA}.{table}: {n} rows ({how})")

    spark.sql(view_sql(args.data_dir))
    rows = spark.table(VIEW).count()
    print(f"{VIEW}: {rows} rows (expected 2240)")
    if rows == 0:
        raise SystemExit("the view is empty")
```

- **Types.** CSV has no types. A `CASTS` table (full file in the appendix) gives each id its `int`, prices their `decimal(10,2)`, and the invoice date (`2021/1/1` in the dump) a timestamp, as in the Postgres `CREATE TABLE`s.
- **Overwrite.** Every run rewrites the tables (`mode("overwrite")`), so a run is always reproducible from the files in the bundle.
- **The schema is created by the task** (`CREATE SCHEMA IF NOT EXISTS workspace.chinook`), not declared in the bundle; Step 2 explains why.

4.3 The view, unchanged

sql/vw_cliente_genero.sql

```
-- Vista para el modelo predictivo: 4 campos
--   tipo_correo : proveedor del correo (texto entre '@' y el primer '.'): gmail, yahoo, hotmail,
embraer...
--   pais        : país del cliente
--   ciudad      : ciudad del cliente
--   genero      : género musical de cada track comprado por el cliente
-- Una fila por track comprado: customer -> invoice -> invoice_line -> track -> genre
-- Misma vista que pipeline-db-ml/db/view.sql, con nombres de tres niveles de Unity Catalog.

CREATE OR REPLACE VIEW workspace.chinook.vw_cliente_genero AS
SELECT
    lower(split_part(split_part(c.email, '@', 2), '.', 1)) AS tipo_correo,
    c.country AS pais,
    c.city    AS ciudad,
    g.name    AS genero
FROM workspace.chinook.customer     c
JOIN workspace.chinook.invoice      i  ON i.customer_id = c.customer_id
JOIN workspace.chinook.invoice_line il ON il.invoice_id = i.invoice_id
```

```
JOIN workspace.chinook.track        t  ON t.track_id    = il.track_id
JOIN workspace.chinook.genre        g  ON g.genre_id    = t.genre_id;
```

Databricks SQL has `split_part` with the same meaning as in Postgres, so the SQL is the same as in the first assignment; only the table names gained the `workspace.chinook.` prefix (and the `\c chinook` psql command disappeared). The task reads this file from the bundle and runs it.

How to check. The task's output in the first run (run 278214313505025):

```
workspace.chinook.genre: 25 rows (spark.read)
workspace.chinook.track: 3503 rows (spark.read)
workspace.chinook.invoice_line: 2240 rows (spark.read)
workspace.chinook.invoice: 412 rows (spark.read)
workspace.chinook.customer: 59 rows (spark.read)
view SQL read from
/Workspace/Users/0188554@up.edu.mx/.bundle/genre_model/dev/files/sql/vw_cliente_genero.sql
workspace.chinook.vw_cliente_genero: 2240 rows (expected 2240)
```

Same counts as the dump, and the same 2 240 view rows as in Postgres. In the workspace you can also open **SQL Editor** and run `SELECT * FROM workspace.chinook.vw_cliente_genero LIMIT 5`.

5. Step 2. The bundle and the Databricks CLI

5.1 Install the CLI and log in

```
# Windows (winget)
winget install Databricks.DatabricksCLI
databricks --version
Databricks CLI v1.20.0

# browser login (OAuth): no token is typed anywhere
databricks auth login --host https://dbc-c19856c3-faa2.cloud.databricks.com
databricks current-user me
```

What. `databricks auth login` opens the browser on the workspace's sign-in page. After you sign in, the browser shows a page saying you are authenticated and can close the tab; the CLI stores an OAuth session in its local cache.

Why. With OAuth there is no long-lived secret to copy or paste: the CLI, and any Python program that uses `databricks-sdk`, reuse that session. The personal access token is created only for the professor (Step 7).

How to check. `databricks current-user me` prints your user (`0188554@up.edu.mx` here).

5.2 The bundle files

databricks.yml

```
# Bundle: daily training job for the music-genre model and its serving endpoint.
#   databricks bundle validate -t dev
#   databricks bundle deploy   -t dev
#   databricks bundle run      -t dev genre_model_train
# Credentials come from the Databricks CLI login (databricks auth login); nothing secret is stored in this
repo.
# The workspace host is public and is set per target below.
bundle:
  name: genre_model

include:
  - resources/*.yml

# The schema workspace.chinook is created by src/load_chinook.py (CREATE SCHEMA IF NOT EXISTS), not as a
# bundle resource: in development mode a bundle schema would be renamed dev_<user>_chinook.

sync:
  include:
    - data/*.csv
    - sql/*.sql

targets:
  dev:
    # No 'mode: development': that mode pauses every schedule and prefixes resource names, and the
    # assignment needs the daily run active under the plain job name.
    default: true
    workspace:
      host: https://dbc-c19856c3-faa2.cloud.databricks.com
```

resources/genre_model_train.job.yml

```
# Daily job: load Chinook into Delta, train the genre model, publish it on the serving endpoint, smoke
test.
# Serverless only (Databricks Free Edition): no clusters, Python tasks with a job-level environment.
resources:
  jobs:
    genre_model_train:
      name: genre_model_train
      max_concurrent_runs: 1
      email_notifications: {}

      schedule:
        quartz_cron_expression: "0 0 3 * * ?"   # every day at 03:00 (Quartz: seconds first)
        timezone_id: America/Mexico_City
        pause_status: UNPAUSED

      environments:
        - environment_key: default
          spec:
            environment_version: "2"
            dependencies:
              - scikit-learn==1.6.1
              - mlflow>=2.15,<4
              - pandas>=1.5,<3
              - databricks-sdk>=0.40

      tasks:
        - task_key: load_chinook
          environment_key: default
          spark_python_task:
            python_file: ../src/load_chinook.py
            parameters:
              - --data-dir
              - ${workspace.file_path}/data

        - task_key: train_model
          depends_on:
            - task_key: load_chinook
          environment_key: default
          spark_python_task:
            python_file: ../src/train_model.py
            parameters:
              - --data-dir
              - ${workspace.file_path}/data
              - --experiment
              - /Users/${workspace.current_user.userName}/genre_model_train

        - task_key: deploy_endpoint
          depends_on:
            - task_key: train_model
          environment_key: default
          timeout_seconds: 3600
          spark_python_task:
            python_file: ../src/deploy_endpoint.py
            parameters:
              - --data-dir
              - ${workspace.file_path}/data

        - task_key: smoke_test
          depends_on:
```

```
      - task_key: deploy_endpoint
    environment_key: default
    spark_python_task:
      python_file: ../src/smoke_test.py
      parameters:
        - --data-dir
        - ${workspace.file_path}/data
```

- `sync.include` makes the CSVs and the SQL file part of what is uploaded, next to the code.
- `${workspace.file_path}` is resolved at deploy time to the folder where the bundle's files land, so the tasks receive the real path of `data/` as a parameter.
- `environments` is how a serverless Python task gets its libraries. scikit-learn is pinned to 1.6.1, the version of the first assignment.
- `timeout_seconds: 3600` on the endpoint task, because creating an endpoint takes minutes.

5.3 Validate and deploy

```
$ databricks bundle validate -t dev
Name: genre_model
Target: dev
Workspace:
  Host: https://dbc-c19856c3-faa2.cloud.databricks.com
  User: 0188554@up.edu.mx
  Path: /Workspace/Users/0188554@up.edu.mx/.bundle/genre_model/dev

Validation OK!

$ databricks bundle deploy -t dev
```

The deploy uploaded the bundle (21 files: code, CSVs, SQL, YAML) to `/Workspace/Users/0188554@up.edu.mx/.bundle/genre_model/dev` and created the job `genre_model_train` (id `740463899927346`): https://dbc-c19856c3-faa2.cloud.databricks.com/jobs/740463899927346?o=7474649054775928.

> **The development-mode trap.**
> Bundle templates put `mode: development` on the `dev` target. That mode is meant for personal experiments: it **pauses every schedule** and prefixes resource names with `[dev <user>]` (and schemas with `dev_<user>_`). The assignment needs the daily training *active* under a plain name, so the target has no `mode` at all (second commit of the repository: "drop development mode so the daily schedule stays active"), and the schema is created by the load task instead of being a bundle resource.

6. Step 3. Training and registering the model

Task 2 is `src/train_model.py`. It is the first assignment's `TrainModel.py` with the Postgres read replaced by a Spark read and the `joblib.dump` replaced by an MLflow registration.

6.1 Read the view, split, measure

src/train_model.py

```
    spark = SparkSession.builder.getOrCreate()
    df = spark.table(VIEW).select(*FEATURES, TARGET).toPandas()
    print(f"Filas recuperadas: {len(df)}")
    if df.empty:
        raise SystemExit(f"{VIEW} is empty")
    print(df.head(3).to_string(index=False))
    print(f"Géneros distintos: {df[TARGET].nunique()}")

    X = df[FEATURES]
    y = df[TARGET]
    # Sin stratify: hay géneros con una sola fila y stratify lanzaría error.
    x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    def nuevo_pipeline():
        return Pipeline([
            ("onehot", OneHotEncoder(handle_unknown="ignore")),
            ("clf", LogisticRegression(max_iter=1000)),
        ])

    mlflow.set_tracking_uri("databricks")
    mlflow.set_registry_uri("databricks-uc")
    if args.experiment:
        mlflow.set_experiment(args.experiment)

    with mlflow.start_run(run_name="genre_model_train") as run:
        model = nuevo_pipeline()
        model.fit(x_train, y_train)
        y_pred = model.predict(x_test)
        accuracy = accuracy_score(y_test, y_pred)
        report = classification_report(y_test, y_pred, zero_division=0)
        print(f"Accuracy (hold-out 20%): {accuracy:.3f}")
        print(report)
        mlflow.log_params({"features": ",".join(FEATURES), "test_size": 0.2, "random_state": 42,
                           "max_iter": 1000, "rows": len(df), "classes": int(y.nunique())})
        mlflow.log_metric("accuracy", accuracy)
        with tempfile.TemporaryDirectory() as tmp:
            path = os.path.join(tmp, "classification_report.txt")
            with open(path, "w", encoding="utf-8") as fh:
                fh.write(f"Accuracy (hold-out 20%): {accuracy:.4f}\n\n{report}")
            mlflow.log_artifact(path)
```

- `spark.table(VIEW)...toPandas()`: 2 240 rows and 3 columns fit easily in memory, so the training itself is plain scikit-learn, exactly as before.
- `set_registry_uri("databricks-uc")` sends model registrations to Unity Catalog (manage-model-lifecycle).

- Inside the MLflow run we log the parameters, the metric `accuracy` and the classification report as a file, so every daily run leaves a comparable record in the experiment `/Users/0188554@up.edu.mx/genre_model_train`.

6.2 The wrapper: same contract as the old API

src/genre_model.py

```
def extraer_tipo_correo(valor) -> str:
    """Same rule as the view vw_cliente_genero (and PredictorRequest in pipeline-db-ml):
    text between '@' and the first '.', lower case. Accepts "gmail" or "ana@yahoo.com.br"."""
    valor = str(valor).strip().lower()
    if "@" in valor:
        valor = valor.split("@", 1)[1]
    return valor.split(".")[0]


def preparar_entrada(model_input) -> pd.DataFrame:
    if not isinstance(model_input, pd.DataFrame):
        model_input = pd.DataFrame(model_input)
    faltan = [c for c in FEATURES if c not in model_input.columns]
    if faltan:
        raise ValueError(f"faltan columnas: {faltan}; se esperan {FEATURES}")
    X = model_input[FEATURES].copy()  # by name: dataframe_records does not guarantee column order
    X["tipo_correo"] = X["tipo_correo"].map(extraer_tipo_correo)
    X["pais"] = X["pais"].astype(str).str.strip()
    X["ciudad"] = X["ciudad"].astype(str).str.strip()
    return X


class GenreModel(_Base):
    def __init__(self, pipeline):
        self.pipeline = pipeline

    def predict(self, context, model_input, params=None):
        X = preparar_entrada(model_input)
        generos = self.pipeline.predict(X)
        probas = self.pipeline.predict_proba(X)
        clases = [str(c) for c in self.pipeline.classes_]
        filas = []
        for genero, fila in zip(generos, probas):
            orden = sorted(range(len(clases)), key=lambda k: fila[k], reverse=True)[:3]
            salida = {"genero": str(genero)}
            for pos in range(3):
                if pos < len(orden):
                    salida[f"top_{pos + 1}"] = clases[orden[pos]]
                    salida[f"p_{pos + 1}"] = round(float(fila[orden[pos]]), 4)
                else:
                    salida[f"top_{pos + 1}"] = ""
                    salida[f"p_{pos + 1}"] = 0.0
            filas.append(salida)
        return pd.DataFrame(filas, columns=OUTPUT_COLUMNS)
```

- `extraer_tipo_correo` is the same rule as the view: the endpoint accepts `"yahoo"` or a full address like `"ana@yahoo.com.br"`, and the model always sees `yahoo`.
- Columns are selected **by name**. The docs warn that `dataframe_records` does not guarantee column order (score-custom-model-endpoints); selecting by name makes the order irrelevant.

- `predict_proba` gives one probability per genre; sorting gives the top 3, rounded to 4 decimals.

6.3 Log, register, move the alias

src/train_model.py

```
        # Reentrenar con todos los datos
        model = nuevo_pipeline()
        model.fit(X, y)

        input_example = pd.DataFrame([
            {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"},
            {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"},
            {"tipo_correo": "hotmail", "pais": "United Kingdom", "ciudad": "London"},
        ])
        pip_requirements = [
            f"mlflow=={mlflow.__version__}",
            f"scikit-learn=={sklearn.__version__}",
            f"pandas=={pd.__version__}",
            f"numpy=={np.__version__}",
        ]
        try:
            import cloudpickle
            pip_requirements.append(f"cloudpickle=={cloudpickle.__version__}")
        except ImportError:
            pass

        log_kwargs = dict(
            python_model=GenreModel(model),
            input_example=input_example,
            code_paths=[os.path.join(here, "genre_model.py")],
            pip_requirements=pip_requirements,
            registered_model_name=MODEL_NAME,
        )
        if int(mlflow.__version__.split(".")[0]) >= 3:
            info = mlflow.pyfunc.log_model(name="model", **log_kwargs)
        else:
            info = mlflow.pyfunc.log_model(artifact_path="model", **log_kwargs)

    client = MlflowClient(registry_uri="databricks-uc")
    version = getattr(info, "registered_model_version", None)
    if version is None:
        versions = [mv for mv in client.search_model_versions(f"name='{MODEL_NAME}'")
                    if mv.run_id == run.info.run_id]
        if not versions:
            raise SystemExit(f"no version of {MODEL_NAME} found for run {run.info.run_id}")
        version = max(int(mv.version) for mv in versions)
    version = str(version)
    client.set_registered_model_alias(MODEL_NAME, ALIAS, version)
    print(f"Registered {MODEL_NAME} version {version}; alias {ALIAS} -> {version}; run {run.info.run_id}")
```

- **Signature.** A model version in Unity Catalog must have a signature. Passing `input_example` lets MLflow infer it by calling `predict` on the example: three string columns in, seven columns out (`genero` , `top_1` , `top_2` , `top_3` strings and `p_1` to `p_3` numbers). The log shows "Inferring model signature from input example".
- **`code_paths`** ships `genre_model.py` with the model, so the serving container can import the class.

- `pip_requirements` pins the exact versions the job trained with, so serving unpickles the pipeline with the same scikit-learn.
- **Alias.** `set_registered_model_alias(..., "Champion", version)` moves the label to the new version. Task 3 reads the alias, not a hard-coded number.

How to check. The task's output in the first run (excerpt):

```
Filas recuperadas: 2240
tipo_correo   pais      ciudad genero
      yahoo  Italy        Rome   Rock
       shaw Canada Yellowknife   Rock
        uol Brazil    Brasília   Rock

Accuracy (hold-out 20%): 0.393
                    precision    recall  f1-score   support
...
             Latin       0.40      0.21      0.27        92
             Metal       0.39      0.39      0.39        46
              Rock       0.39      0.88      0.54       158
...
          accuracy                           0.39       448

2026/10/10 04:23:35 INFO mlflow.pyfunc: Inferring model signature from input example
Successfully registered model 'workspace.chinook.genero_cliente'.
Created version '1' of model 'workspace.chinook.genero_cliente': https://dbc-c19856c3-
faa2.cloud.databricks.com/explore/data/models/workspace/chinook/genero_cliente/version/1?
o=7474649054775928
Registered workspace.chinook.genero_cliente version 1; alias Champion -> 1; run
1d38d577f7e943659dc8a9eea2bef1c0
task value model_version set
```

The model page: Catalog → workspace → chinook → genero_cliente → version 1; the experiment: experiment 4227171573263053.

About the accuracy. The first assignment printed 0.353 on EC2; here it is 0.393 (0.39). Same data, same pipeline, same `random_state`. The difference is the **row order**: Postgres and Spark return the view's rows in different orders, and `train_test_split` shuffles by position, so the 448 test rows are a different sample. For reference, always answering "Rock" would be right on 37 % of the rows; the model gets there and also learns the exceptions (Berlin → Metal).

7. Step 4. The serving endpoint

Task 3, `src/deploy_endpoint.py`, publishes the `Champion` version on the endpoint `genero-cliente`. Authentication is the job's own identity (`WorkspaceClient()` with no arguments), so the code holds no secret.

src/deploy_endpoint.py (create or update)

```
    entity = ServedEntityInput(
        entity_name=MODEL_NAME,
        entity_version=version,
        workload_size="Small",
        scale_to_zero_enabled=True,
    )

    try:
        existing = w.serving_endpoints.get(ENDPOINT)
    except (NotFound, ResourceDoesNotExist):
        existing = None

    if existing is None:
        print(f"creating endpoint {ENDPOINT} with version {version} (waiting up to {TIMEOUT})")
        endpoint = w.serving_endpoints.create_and_wait(
            name=ENDPOINT,
            config=EndpointCoreConfigInput(name=ENDPOINT, served_entities=[entity]),
            timeout=TIMEOUT,
        )
    else:
        served = [(e.entity_name, str(e.entity_version))
                  for e in ((existing.config.served_entities if existing.config else None) or [])]
        updating = existing.state and existing.state.config_update and \
            existing.state.config_update.value == "IN_PROGRESS"
        if updating:
            print(f"endpoint {ENDPOINT} is updating; waiting for it first")
            existing = w.serving_endpoints.wait_get_serving_endpoint_not_updating(ENDPOINT,
timeout=TIMEOUT)
            served = [(e.entity_name, str(e.entity_version))
                      for e in ((existing.config.served_entities if existing.config else None) or [])]
        if served == [(MODEL_NAME, version)]:
            print(f"endpoint {ENDPOINT} already serves version {version}; no update")
            endpoint = existing
        else:
            print(f"updating endpoint {ENDPOINT}: {served} -> {MODEL_NAME} v{version} (waiting up to
{TIMEOUT})")
            endpoint = w.serving_endpoints.update_config_and_wait(
                name=ENDPOINT,
                served_entities=[entity],
                timeout=TIMEOUT,
            )
```

Create vs update. The first run finds no endpoint and creates it with one served entity: model `workspace.chinook.genero_cliente`, the alias's version, size `Small`, scale to zero on. Every later run finds it and calls `update_config` with the new version. If the endpoint already serves that version (for example when a run is repeated after a failure in task 4), it does nothing.

No downtime. During an update the old configuration keeps serving until the new one is ready (create-manage-serving-endpoints). The `_and_wait` calls block until that happens, up to 30 minutes.

Why the job owns the endpoint. A bundle can also declare an endpoint as a resource, but its model version would then be fixed in YAML and every deploy would push it back, fighting the job's daily update. Letting the job create or update the endpoint means one run trains *and* publishes, with no manual step in the UI.

> **How to check.** The task's output in the first run:
>
> ```
> workspace.chinook.genero_cliente@Champion = version 1
> creating endpoint genero-cliente with version 1 (waiting up to 0:30:00)
> endpoint genero-cliente: ready=READY config_update=NOT_UPDATING
>   serves workspace.chinook.genero_cliente version 1
> invocation URL: https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-
> cliente/invocations
> ```
>
> At any time, from your machine:
>
> ```
> databricks serving-endpoints get genero-cliente
> ```
>
> The fields to read are `state.ready` (`READY`), `state.config_update` (`NOT_UPDATING` , or `IN_PROGRESS` while a new version is being prepared) and `config.served_entities[].entity_version` . In the UI: **Serving** → `genero-cliente` .

The whole first run took about 12 minutes (22:20:54 to 22:32:38), almost all of it the endpoint creation: Databricks builds a container image with the model's dependencies the first time.

8. Step 5. The smoke test and querying from outside

8.1 Task 4: the job tests its own endpoint

src/smoke_test.py

```
ROWS = [
    {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"},
    {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"},
    {"tipo_correo": "ana@yahoo.com.br", "pais": "Brazil", "ciudad": "São Paulo"},
]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", default=None)  # accepted for symmetry with the other tasks
    ap.parse_args()

    from databricks.sdk import WorkspaceClient

    w = WorkspaceClient()
    response = w.serving_endpoints.query(name=ENDPOINT, dataframe_records=ROWS)
    predictions = response.predictions or []
    for row, pred in zip(ROWS, predictions):
        print(json.dumps({"entrada": row, "prediccion": pred}, ensure_ascii=False))
```

The job queries the endpoint it just published with three known rows and fails if any prediction is missing. A green run therefore means "the model was trained **and** the endpoint answers", not just "the script finished". The third row checks the email normalisation: a full address must behave like its provider.

```
{"entrada": {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"}, "prediccion": {"genero":
"Metal", "top_1": "Metal", "p_1": 0.4076, "top_2": "Rock", "p_2": 0.3364, "top_3": "Alternative & Punk",
"p_3": 0.0752}}
{"entrada": {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"}, "prediccion": {"genero":
"Rock", "top_1": "Rock", "p_1": 0.515, "top_2": "Latin", "p_2": 0.2183, "top_3": "Alternative & Punk",
"p_3": 0.0942}}
{"entrada": {"tipo_correo": "ana@yahoo.com.br", "pais": "Brazil", "ciudad": "São Paulo"}, "prediccion":
{"genero": "Rock", "top_1": "Rock", "p_1": 0.398, "top_2": "Latin", "p_2": 0.2572, "top_3": "Alternative &
Punk", "p_3": 0.1059}}
yahoo/Germany/Berlin -> Metal (Metal, as the previous model)
smoke test passed
```

8.2 The request and the answer

URL: `https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-cliente/invocations`, method `POST`, header `Authorization: Bearer <TOKEN>`, body `{"dataframe_records": [...]}`, answer `{"predictions": [...]}` (score-custom-model-endpoints).

curl (bash)

```
curl -X POST https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-cliente/invocations \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"dataframe_records": [{"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"}]}'
```

```
{
  "predictions": [
    {
      "genero": "Metal",
      "top_1": "Metal",
      "p_1": 0.4076,
      "top_2": "Rock",
      "p_2": 0.3364,
      "top_3": "Alternative & Punk",
      "p_3": 0.0752
    }
  ]
}
```

Berlin is the same good example as in the first assignment: the answer is *not* the majority class, and the probabilities are identical to the EC2 API's (0.4076, 0.3364, 0.0752), because the final model is trained on all rows and row order does not change a full-data fit.

8.3 From Python

The minimal version, as in the README: the token comes from the environment variable `DATABRICKS_TOKEN`, never from the code.

README.md (Python example)

```
import os
import requests

url = "https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-cliente/invocations"
headers = {
    "Authorization": f"Bearer {os.environ['DATABRICKS_TOKEN']}",
    "Content-Type": "application/json",
}
body = {"dataframe_records": [
    {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"},
    {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"},
]}

r = requests.post(url, headers=headers, json=body, timeout=120)
r.raise_for_status()
for p in r.json()["predictions"]:
    print(p["genero"], p["top_1"], p["p_1"], p["top_2"], p["p_2"], p["top_3"], p["p_3"])
```

```
Metal Metal 0.4076 Rock 0.3364 Alternative & Punk 0.0752
Rock Rock 0.515 Latin 0.2183 Alternative & Punk 0.0942
```

The repository also has a ready-made client, `scripts/query_endpoint.py`. Its only interesting part is how it chooses the credentials:

scripts/query_endpoint.py (authentication)

```
    token = os.environ.get("DATABRICKS_TOKEN")
    if token:
        host = os.environ.get("DATABRICKS_HOST")
        if not host:
            print("DATABRICKS_TOKEN is set but DATABRICKS_HOST is not", file=sys.stderr)
```

```
            return 2
        headers = {"Authorization": f"Bearer {token}"}
        auth = "DATABRICKS_TOKEN"
    else:
        try:
            from databricks.sdk import WorkspaceClient
        except ImportError:
            print("no DATABRICKS_TOKEN and databricks-sdk is not installed (pip install databricks-sdk)",
                  file=sys.stderr)
            return 2
        try:
            w = WorkspaceClient(profile=args.profile) if args.profile else WorkspaceClient()
            host = w.config.host
            headers = dict(w.config.authenticate())
        except Exception as e:  # noqa: BLE001 - report the auth failure plainly
            print(f"databricks-sdk auth failed ({type(e).__name__}): {e}", file=sys.stderr)
            print("set DATABRICKS_HOST and DATABRICKS_TOKEN, run `databricks auth login`,"
                  " or pass --profile", file=sys.stderr)
            return 2
        source = f"profile {args.profile}" if args.profile else "default auth"
        auth = f"databricks-sdk ({source}, {w.config.auth_type})"
```

With `DATABRICKS_TOKEN` set it sends the `Bearer` header; without it, `databricks-sdk` authenticates (a named CLI profile with `--profile`, or the session of `databricks auth login` plus `DATABRICKS_HOST`; for the latter the `databricks` executable must be on the `PATH`). Real run on October 9, 2026, with no token set:

```
$ export DATABRICKS_HOST=https://dbc-c19856c3-faa2.cloud.databricks.com
$ .venv/Scripts/python scripts/query_endpoint.py
auth: databricks-sdk (default auth, databricks-cli)
POST https://dbc-c19856c3-faa2.cloud.databricks.com/serving-endpoints/genero-cliente/invocations -> HTTP
200
{
  "predictions": [
    {
      "genero": "Metal",
      "top_1": "Metal",
      "p_1": 0.4076,
      "top_2": "Rock",
      "p_2": 0.3364,
      "top_3": "Alternative & Punk",
      "p_3": 0.0752
    }
  ]
}
```

8.4 From the Databricks CLI

```
$ databricks serving-endpoints query genero-cliente --json '{"dataframe_records": [{"tipo_correo":
"gmail", "pais": "Brazil", "ciudad": "São Paulo"}]}'
{
  "predictions": [
    {
      "genero": "Rock",
      "p_1": 0.515,
      "p_2": 0.2183,
      "p_3": 0.0942,
      "top_1": "Rock",
      "top_2": "Latin",
```

```
      "top_3": "Alternative \u0026 Punk"
    }
  ]
}
```

The CLI prints keys in alphabetical order and escapes `&` as `\u0026`; the values are the same. The docs recommend `dataframe_split` when column order matters; our wrapper selects by name, so both formats work:

```
$ databricks serving-endpoints query genero-cliente --json '{"dataframe_split": {"columns":
["tipo_correo", "pais", "ciudad"], "data": [["yahoo", "Germany", "Berlin"], ["hotmail", "United Kingdom",
"London"]]}}'
{
  "predictions": [
    {
      "genero": "Metal",
      "p_1": 0.4076,
      "p_2": 0.3364,
      "p_3": 0.0752,
      "top_1": "Metal",
      "top_2": "Rock",
      "top_3": "Alternative \u0026 Punk"
    },
    {
      "genero": "Latin",
      "p_1": 0.3595,
      "p_2": 0.2372,
      "p_3": 0.1856,
      "top_1": "Latin",
      "top_2": "Rock",
      "top_3": "Metal"
    }
  ]
}
```

9. Step 6. The schedule and what each run does

resources/genre_model_train.job.yml (schedule)

```
    schedule:
      quartz_cron_expression: "0 0 3 * * ?"   # every day at 03:00 (Quartz: seconds first)
      timezone_id: America/Mexico_City
      pause_status: UNPAUSED
```

Field	second	minute	hour	day of month	month	day of week
Value	`0`	`0`	`3`	`*`	`*`	`?`
Meaning	at :00	at :00	03 h	every day	every month	no specific day

Why daily and not hourly. The Free Edition fair-use quota switches compute off for the rest of the day when it is exceeded. Chinook never changes, so even a daily retrain is generous; it demonstrates the periodic pipeline the assignment asks for while keeping well inside the quota. The time zone is explicit (`America/Mexico_City`), so 03:00 means 03:00 in Mexico City.

Why `UNPAUSED` is written out: it documents the intent and, together with the absence of development mode, guarantees the schedule is live after a deploy (bundles/examples).

How to check.

```
$ databricks jobs get 740463899927346 -o json      (excerpt)
{
  "job_id": 740463899927346,
  "settings": {
    "name": "genre_model_train",
    "schedule": {
      "pause_status": "UNPAUSED",
      "quartz_cron_expression": "0 0 3 * * ?",
      "timezone_id": "America/Mexico_City"
    }
  }
}
```

9.1 What every run does

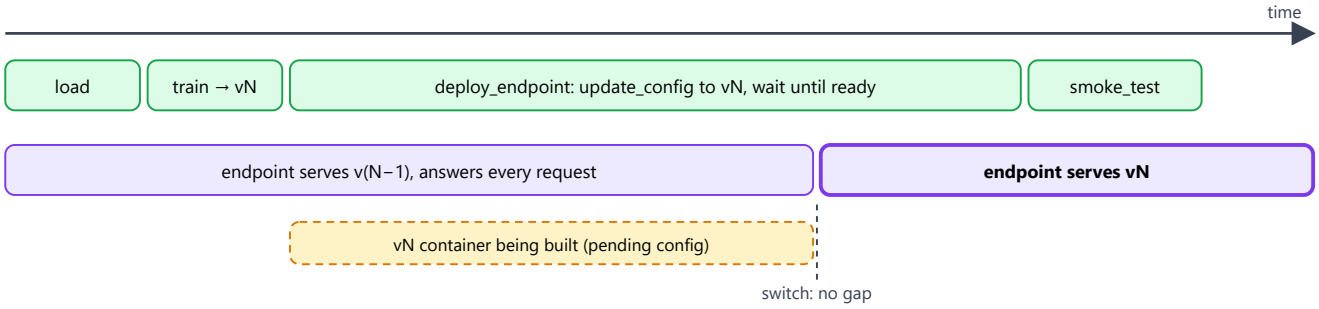


Figure 3. One run. Version N is registered in seconds; the endpoint keeps answering with the previous version while N is prepared, then switches.

Each run registers a new version N of `workspace.chinook.genero_cliente`, moves `Champion` to N, and updates the endpoint to N. The URL and the token never change, so the professor's example keeps working across retrains. This was visible while the second run (started by hand right after the first) was updating: the endpoint reported `ready: READY` and `config_update: IN_PROGRESS`, still serving version 1, with version 2 listed as the pending configuration (`Container creation pending.`), and queries kept returning HTTP 200.

Because the data does not change, version N predicts the same as version N−1. In a real store with new purchases every day, the same job would pick them up without any change to the code.

10. Step 7. The token and the delivery to the professor

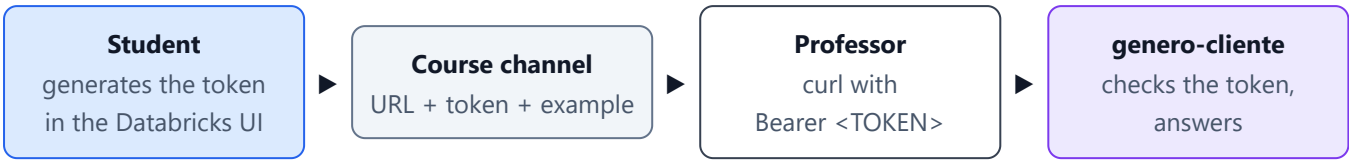


Figure 4. The token goes from the Databricks UI to the professor directly. It is never in the repository, in this document, or in any file under version control.

10.1 How to generate it

1. In the workspace, click your username (top right) → **Settings** → **Developer**.
2. Next to **Access tokens**, click **Manage** → **Generate new token**.
3. Comment (name): `profesor-bases-de-datos`. Lifetime: `60` days. If the dialog offers a scope, choose "Other APIs" and limit it to serving endpoints.
4. Click **Generate** and copy the token **now**: it is shown only once.
5. Send it to the professor through the course channel together with the URL and the curl example.

Source: https://docs.databricks.com/aws/en/dev-tools/auth/pat ("Save the token securely and don't share it").

10.2 What it grants

A personal access token acts as its owner. Unscoped, it can call any Databricks REST API the owner can, in this workspace only: that is why the lifetime is short, why a scope is used when the UI offers one, and why it is shared only with the professor. On this endpoint it is sent as `Authorization: Bearer <TOKEN>`; requests without a valid token are rejected. Personal access tokens work on standard endpoints like this one; route-optimized endpoints would require OAuth instead.

10.3 How to revoke it

Same page (**Settings** → **Developer** → **Access tokens** → **Manage**): revoke the token `profesor-bases-de-datos`. It also expires by itself after 60 days, and Databricks revokes any token unused for 90 days. Revoking it does not affect the job or the endpoint, which use their own identity.

10.4 Why it can never leak through the repository

`.gitignore` (first lines)

```
# secrets: the token lives only in your environment or the Databricks CLI profile
.env
.env.*
*.pem
```

The code never reads a token from a file: the job uses its ambient identity, the local client reads `DATABRICKS_TOKEN` from the environment or uses the CLI's OAuth session, and every document shows the literal placeholder `<TOKEN>`.

11. Operating it: runs, costs, scale to zero, tearing down

11.1 Run it by hand and see the runs

```
# from the repository folder: start a run and stream its result
databricks bundle run -t dev genre_model_train

# list the runs of the job
databricks jobs list-runs --job-id 740463899927346
```

In the UI: open the job's page and click **Run now**; the runs list on the same page shows each run with its duration, and opening one shows the output of each task (the excerpts in this document come from there). The first run: run 278214313505025.

11.2 Costs and the Free Edition quota

- Free Edition has no bill. The limit is the **fair-use quota**: exceeding it switches compute off for the rest of the day. One run a day of a few short tasks stays far below it.
- The expensive part of a run is waiting for the endpoint to be built, not our own compute: training takes seconds.
- Free Edition allows a limited number of active serving endpoints (the number is not published) and no GPU; this project uses one CPU endpoint of size `Small`.
- Per-request limits of Model Serving (payload size, execution time) are listed in model-serving-limits; a request with a few rows is far below them.

11.3 Scale to zero

With no traffic, the endpoint releases its compute. The next request wakes it up and takes noticeably longer than the following ones; a client should use a generous timeout (the local client uses 120 seconds). If the professor's first call is slow, a second call answers at normal speed.

11.4 Tearing it down (described, not run)

```
# 1. remove what the bundle created: the job and the uploaded files
databricks bundle destroy -t dev

# 2. the endpoint was created by the job, not by the bundle, so delete it explicitly
databricks serving-endpoints delete genero-cliente

# 3. optional, in the SQL editor: tables, view and registered model live in the schema
DROP SCHEMA workspace.chinook CASCADE;

# 4. revoke the professor's token (Settings > Developer > Access tokens)
```

Order matters: destroying the bundle first stops the schedule, so no run recreates the endpoint after step 2. None of these steps were run; the project stays live for grading.

12. Appendix. Full source of the four task files and the wrapper

src/load_chinook.py

```
"""Task 1: load the five Chinook CSVs into Delta tables and (re)create the view vw_cliente_genero.

Reads <data-dir>/{genre,track,invoice_line,invoice,customer}.csv (bundle workspace files), writes
workspace.chinook.<table> (overwrite) and runs sql/vw_cliente_genero.sql.
"""
import argparse
import os

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

CATALOG = "workspace"
SCHEMA = "chinook"
VIEW = f"{CATALOG}.{SCHEMA}.vw_cliente_genero"

# Column types from the CREATE TABLE statements of the Postgres dump; everything else stays string.
CASTS = {
    "genre": {"genre_id": "int"},
    "track": {"track_id": "int", "album_id": "int", "media_type_id": "int", "genre_id": "int",
              "milliseconds": "int", "bytes": "int", "unit_price": "decimal(10,2)"},
    "invoice_line": {"invoice_line_id": "int", "invoice_id": "int", "track_id": "int",
                     "unit_price": "decimal(10,2)", "quantity": "int"},
    "invoice": {"invoice_id": "int", "customer_id": "int", "total": "decimal(10,2)"},
    "customer": {"customer_id": "int", "support_rep_id": "int"},
}
TABLES = list(CASTS)

VIEW_SQL_FALLBACK = f"""
CREATE OR REPLACE VIEW {VIEW} AS
SELECT
    lower(split_part(split_part(c.email, '@', 2), '.', 1)) AS tipo_correo,
    c.country AS pais,
    c.city    AS ciudad,
    g.name    AS genero
FROM {CATALOG}.{SCHEMA}.customer     c
JOIN {CATALOG}.{SCHEMA}.invoice      i  ON i.customer_id = c.customer_id
JOIN {CATALOG}.{SCHEMA}.invoice_line il ON il.invoice_id = i.invoice_id
JOIN {CATALOG}.{SCHEMA}.track        t  ON t.track_id    = il.track_id
JOIN {CATALOG}.{SCHEMA}.genre        g  ON g.genre_id    = t.genre_id
"""


def read_csv(spark, path):
    """spark.read first; if Spark cannot reach workspace files, read with pandas and convert."""
    try:
        df = (spark.read.option("header", "true").option("inferSchema", "false")
              .option("quote", '"').option("escape", '"').option("encoding", "UTF-8")
              .csv(path))
        df.limit(1).collect()
        return df, "spark.read"
    except Exception as e:  # noqa: BLE001 - fall back on any read failure
        print(f"spark.read failed for {path} ({type(e).__name__}: {str(e)[:200]}); using pandas")
    import pandas as pd
    pdf = pd.read_csv(path, dtype=str, keep_default_na=False, encoding="utf-8")
    pdf = pdf.astype(object).where(pdf != "", None)
    schema = ", ".join(f"`{c}` string" for c in pdf.columns)
    return spark.createDataFrame(pdf.values.tolist(), schema=schema), "pandas"
```

```
def typed(table, df):
    for col, typ in CASTS[table].items():
        df = df.withColumn(col, F.col(col).cast(typ))
    if table == "invoice":
        # the dump writes dates as 2021/1/1
        df = df.withColumn("invoice_date", F.to_timestamp(F.col("invoice_date"), "yyyy/M/d"))
    return df


def view_sql(data_dir):
    """Read sql/vw_cliente_genero.sql next to the bundle's src/ and data/; inline copy as fallback."""
    candidates = []
    try:
        candidates.append(os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                                       "sql", "vw_cliente_genero.sql"))
    except NameError:  # __file__ is not always defined in a job's python task
        pass
    candidates.append(os.path.join(os.path.dirname(data_dir.rstrip("/")), "sql", "vw_cliente_genero.sql"))
    for path in candidates:
        try:
            with open(path, encoding="utf-8") as fh:
                text = fh.read()
            print(f"view SQL read from {path}")
            break
        except OSError as e:
            print(f"cannot read {path}: {e}")
    else:
        print("view SQL: using the inlined copy")
        return VIEW_SQL_FALLBACK.strip()
    lines = [ln for ln in text.splitlines() if not ln.strip().startswith("--")]
    return "\n".join(lines).strip().rstrip(";").strip()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", required=True)
    args = ap.parse_args()

    spark = SparkSession.builder.getOrCreate()
    spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SCHEMA}")

    for table in TABLES:
        path = f"{args.data_dir.rstrip('/')}/{table}.csv"
        raw, how = read_csv(spark, path)
        df = typed(table, raw)
        (df.write.format("delta").mode("overwrite").option("overwriteSchema", "true")
           .saveAsTable(f"{CATALOG}.{SCHEMA}.{table}"))
        n = spark.table(f"{CATALOG}.{SCHEMA}.{table}").count()
        print(f"{CATALOG}.{SCHEMA}.{table}: {n} rows ({how})")

    spark.sql(view_sql(args.data_dir))
    rows = spark.table(VIEW).count()
    print(f"{VIEW}: {rows} rows (expected 2240)")
    if rows == 0:
        raise SystemExit("the view is empty")


if __name__ == "__main__":
    main()
```

src/train_model.py

```
"""Task 2: train the genre model from the view and register it in Unity Catalog.
```

```
Same features, target, pipeline and hold-out as pipeline-db-ml/src/contexts/train_model/TrainModel.py.
Logs accuracy and the classification report to MLflow, refits on all rows, logs the pyfunc wrapper
(GenreModel) as workspace.chinook.genero_cliente and points the alias Champion at the new version.
"""
import argparse
import os
import sys
import tempfile

VIEW = "workspace.chinook.vw_cliente_genero"
MODEL_NAME = "workspace.chinook.genero_cliente"
ALIAS = "Champion"


def src_dir(data_dir):
    try:
        return os.path.dirname(os.path.abspath(__file__))
    except NameError:  # __file__ is not always defined in a job's python task
        return os.path.join(os.path.dirname(data_dir.rstrip("/")), "src")


def get_dbutils(spark):
    try:
        from databricks.sdk.runtime import dbutils  # noqa: F401
        return dbutils
    except Exception:  # noqa: BLE001
        pass
    try:
        from pyspark.dbutils import DBUtils
        return DBUtils(spark)
    except Exception:  # noqa: BLE001
        return None


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", required=True)
    ap.add_argument("--experiment", default=None, help="MLflow experiment path, e.g.
/Users/<me>/genre_model_train")
    args = ap.parse_args()

    here = src_dir(args.data_dir)
    if here not in sys.path:
        sys.path.insert(0, here)
    from genre_model import FEATURES, TARGET, GenreModel  # noqa: E402

    import mlflow
    import numpy as np
    import pandas as pd
    import sklearn
    from mlflow.tracking import MlflowClient
    from pyspark.sql import SparkSession
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import accuracy_score, classification_report
    from sklearn.model_selection import train_test_split
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import OneHotEncoder

    spark = SparkSession.builder.getOrCreate()
    df = spark.table(VIEW).select(*FEATURES, TARGET).toPandas()
    print(f"Filas recuperadas: {len(df)}")
    if df.empty:
        raise SystemExit(f"{VIEW} is empty")
    print(df.head(3).to_string(index=False))
    print(f"Géneros distintos: {df[TARGET].nunique()}")
```

```
X = df[FEATURES]
y = df[TARGET]
# Sin stratify: hay géneros con una sola fila y stratify lanzaría error.
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

def nuevo_pipeline():
    return Pipeline([
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
        ("clf", LogisticRegression(max_iter=1000)),
    ])

mlflow.set_tracking_uri("databricks")
mlflow.set_registry_uri("databricks-uc")
if args.experiment:
    mlflow.set_experiment(args.experiment)

with mlflow.start_run(run_name="genre_model_train") as run:
    model = nuevo_pipeline()
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, zero_division=0)
    print(f"Accuracy (hold-out 20%): {accuracy:.3f}")
    print(report)
    mlflow.log_params({"features": ",".join(FEATURES), "test_size": 0.2, "random_state": 42,
                       "max_iter": 1000, "rows": len(df), "classes": int(y.nunique())})
    mlflow.log_metric("accuracy", accuracy)
    with tempfile.TemporaryDirectory() as tmp:
        path = os.path.join(tmp, "classification_report.txt")
        with open(path, "w", encoding="utf-8") as fh:
            fh.write(f"Accuracy (hold-out 20%): {accuracy:.4f}\n\n{report}")
        mlflow.log_artifact(path)

    # Reentrenar con todos los datos
    model = nuevo_pipeline()
    model.fit(X, y)

    input_example = pd.DataFrame([
        {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"},
        {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"},
        {"tipo_correo": "hotmail", "pais": "United Kingdom", "ciudad": "London"},
    ])
    pip_requirements = [
        f"mlflow=={mlflow.__version__}",
        f"scikit-learn=={sklearn.__version__}",
        f"pandas=={pd.__version__}",
        f"numpy=={np.__version__}",
    ]
    try:
        import cloudpickle
        pip_requirements.append(f"cloudpickle=={cloudpickle.__version__}")
    except ImportError:
        pass

    log_kwargs = dict(
        python_model=GenreModel(model),
        input_example=input_example,
        code_paths=[os.path.join(here, "genre_model.py")],
        pip_requirements=pip_requirements,
        registered_model_name=MODEL_NAME,
    )
    if int(mlflow.__version__.split(".")[0]) >= 3:
        info = mlflow.pyfunc.log_model(name="model", **log_kwargs)
    else:
```

```
            info = mlflow.pyfunc.log_model(artifact_path="model", **log_kwargs)

    client = MlflowClient(registry_uri="databricks-uc")
    version = getattr(info, "registered_model_version", None)
    if version is None:
        versions = [mv for mv in client.search_model_versions(f"name='{MODEL_NAME}'")
                    if mv.run_id == run.info.run_id]
        if not versions:
            raise SystemExit(f"no version of {MODEL_NAME} found for run {run.info.run_id}")
        version = max(int(mv.version) for mv in versions)
    version = str(version)
    client.set_registered_model_alias(MODEL_NAME, ALIAS, version)
    print(f"Registered {MODEL_NAME} version {version}; alias {ALIAS} -> {version}; run {run.info.run_id}")

    dbutils = get_dbutils(spark)
    try:
        dbutils.jobs.taskValues.set(key="model_version", value=version)
        print("task value model_version set")
    except Exception as e:  # noqa: BLE001 - the next task falls back to the alias
        print(f"task value not set ({type(e).__name__}); deploy_endpoint will resolve the alias")


if __name__ == "__main__":
    main()
```

src/genre_model.py

```
"""MLflow pyfunc wrapper around the scikit-learn pipeline (OneHotEncoder -> LogisticRegression).

Input: a DataFrame with columns tipo_correo, pais, ciudad (tipo_correo may be a full email).
Output: a DataFrame with genero, top_1, p_1, top_2, p_2, top_3, p_3 (same contract as the
POST /api/model of pipeline-db-ml: the predicted genre plus the top 3 with probabilities).

Logged with code_paths=[this file], so the serving container can import it as `genre_model`.
"""
import pandas as pd

try:  # mlflow is not needed to fit or test the pipeline locally
    from mlflow.pyfunc import PythonModel as _Base
except ImportError:  # pragma: no cover
    _Base = object

FEATURES = ["tipo_correo", "pais", "ciudad"]
TARGET = "genero"
OUTPUT_COLUMNS = ["genero", "top_1", "p_1", "top_2", "p_2", "top_3", "p_3"]


def extraer_tipo_correo(valor) -> str:
    """Same rule as the view vw_cliente_genero (and PredictorRequest in pipeline-db-ml):
    text between '@' and the first '.', lower case. Accepts "gmail" or "ana@yahoo.com.br"."""
    valor = str(valor).strip().lower()
    if "@" in valor:
        valor = valor.split("@", 1)[1]
    return valor.split(".")[0]


def preparar_entrada(model_input) -> pd.DataFrame:
    if not isinstance(model_input, pd.DataFrame):
        model_input = pd.DataFrame(model_input)
    faltan = [c for c in FEATURES if c not in model_input.columns]
    if faltan:
        raise ValueError(f"faltan columnas: {faltan}; se esperan {FEATURES}")
    X = model_input[FEATURES].copy()  # by name: dataframe_records does not guarantee column order
    X["tipo_correo"] = X["tipo_correo"].map(extraer_tipo_correo)
```

```
    X["pais"] = X["pais"].astype(str).str.strip()
    X["ciudad"] = X["ciudad"].astype(str).str.strip()
    return X


class GenreModel(_Base):
    def __init__(self, pipeline):
        self.pipeline = pipeline

    def predict(self, context, model_input, params=None):
        X = preparar_entrada(model_input)
        generos = self.pipeline.predict(X)
        probas = self.pipeline.predict_proba(X)
        clases = [str(c) for c in self.pipeline.classes_]
        filas = []
        for genero, fila in zip(generos, probas):
            orden = sorted(range(len(clases)), key=lambda k: fila[k], reverse=True)[:3]
            salida = {"genero": str(genero)}
            for pos in range(3):
                if pos < len(orden):
                    salida[f"top_{pos + 1}"] = clases[orden[pos]]
                    salida[f"p_{pos + 1}"] = round(float(fila[orden[pos]]), 4)
                else:
                    salida[f"top_{pos + 1}"] = ""
                    salida[f"p_{pos + 1}"] = 0.0
            filas.append(salida)
        return pd.DataFrame(filas, columns=OUTPUT_COLUMNS)
```

src/deploy_endpoint.py

```
"""Task 3: publish the Champion version of workspace.chinook.genero_cliente on the endpoint genero-cliente.

Creates the endpoint (Small, scale to zero) if it does not exist, otherwise updates its config to the
new version; waits until it is ready. Auth is the job's ambient identity (WorkspaceClient()).
The old config keeps serving until the new one is ready, so a daily update has no downtime.
"""
import argparse
from datetime import timedelta

ENDPOINT = "genero-cliente"
MODEL_NAME = "workspace.chinook.genero_cliente"
ALIAS = "Champion"
TIMEOUT = timedelta(minutes=30)


def version_from_task_value():
    try:
        from databricks.sdk.runtime import dbutils
        v = dbutils.jobs.taskValues.get(taskKey="train_model", key="model_version", debugValue="")
        return str(v) if v else None
    except Exception as e:  # noqa: BLE001
        print(f"task value not available ({type(e).__name__})")
        return None


def champion_version():
    import mlflow
    from mlflow.tracking import MlflowClient
    mlflow.set_registry_uri("databricks-uc")
    mv = MlflowClient(registry_uri="databricks-uc").get_model_version_by_alias(MODEL_NAME, ALIAS)
    return str(mv.version)


def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", default=None)  # accepted for symmetry with the other tasks
    ap.parse_args()

    from databricks.sdk import WorkspaceClient
    from databricks.sdk.errors import NotFound, ResourceDoesNotExist
    from databricks.sdk.service.serving import EndpointCoreConfigInput, ServedEntityInput

    w = WorkspaceClient()

    alias_version = champion_version()
    task_version = version_from_task_value()
    if task_version and task_version != alias_version:
        print(f"note: task value says version {task_version}, alias {ALIAS} says {alias_version}; using the
alias")
    version = alias_version
    print(f"{MODEL_NAME}@{ALIAS} = version {version}")

    entity = ServedEntityInput(
        entity_name=MODEL_NAME,
        entity_version=version,
        workload_size="Small",
        scale_to_zero_enabled=True,
    )

    try:
        existing = w.serving_endpoints.get(ENDPOINT)
    except (NotFound, ResourceDoesNotExist):
        existing = None

    if existing is None:
        print(f"creating endpoint {ENDPOINT} with version {version} (waiting up to {TIMEOUT})")
        endpoint = w.serving_endpoints.create_and_wait(
            name=ENDPOINT,
            config=EndpointCoreConfigInput(name=ENDPOINT, served_entities=[entity]),
            timeout=TIMEOUT,
        )
    else:
        served = [(e.entity_name, str(e.entity_version))
                  for e in ((existing.config.served_entities if existing.config else None) or [])]
        updating = existing.state and existing.state.config_update and \
            existing.state.config_update.value == "IN_PROGRESS"
        if updating:
            print(f"endpoint {ENDPOINT} is updating; waiting for it first")
            existing = w.serving_endpoints.wait_get_serving_endpoint_not_updating(ENDPOINT, timeout=TIMEOUT)
            served = [(e.entity_name, str(e.entity_version))
                      for e in ((existing.config.served_entities if existing.config else None) or [])]
        if served == [(MODEL_NAME, version)]:
            print(f"endpoint {ENDPOINT} already serves version {version}; no update")
            endpoint = existing
        else:
            print(f"updating endpoint {ENDPOINT}: {served} -> {MODEL_NAME} v{version} (waiting up to
{TIMEOUT})")
            endpoint = w.serving_endpoints.update_config_and_wait(
                name=ENDPOINT,
                served_entities=[entity],
                timeout=TIMEOUT,
            )

    state = endpoint.state
    ready = state.ready.value if state and state.ready else None
    config_update = state.config_update.value if state and state.config_update else None
    print(f"endpoint {ENDPOINT}: ready={ready} config_update={config_update}")
    for e in (endpoint.config.served_entities if endpoint.config else None) or []:
        print(f"  serves {e.entity_name} version {e.entity_version}")
```

```
    host = w.config.host.rstrip("/")
    print(f"invocation URL: {host}/serving-endpoints/{ENDPOINT}/invocations")
    if ready != "READY":
        raise SystemExit(f"endpoint {ENDPOINT} is not READY (ready={ready})")


if __name__ == "__main__":
    main()
```

src/smoke_test.py

```
"""Task 4: query the endpoint genero-cliente with three known rows; fail the job if it does not answer."""
import argparse
import json
import sys

ENDPOINT = "genero-cliente"
ROWS = [
    {"tipo_correo": "yahoo", "pais": "Germany", "ciudad": "Berlin"},
    {"tipo_correo": "gmail", "pais": "Brazil", "ciudad": "São Paulo"},
    {"tipo_correo": "ana@yahoo.com.br", "pais": "Brazil", "ciudad": "São Paulo"},
]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data-dir", default=None)  # accepted for symmetry with the other tasks
    ap.parse_args()

    from databricks.sdk import WorkspaceClient

    w = WorkspaceClient()
    response = w.serving_endpoints.query(name=ENDPOINT, dataframe_records=ROWS)
    predictions = response.predictions or []
    for row, pred in zip(ROWS, predictions):
        print(json.dumps({"entrada": row, "prediccion": pred}, ensure_ascii=False))

    if len(predictions) != len(ROWS):
        print(f"expected {len(ROWS)} predictions, got {len(predictions)}: {predictions!r}")
        sys.exit(1)
    empty = [i for i, p in enumerate(predictions) if not (isinstance(p, dict) and p.get("genero"))]
    if empty:
        print(f"empty genero in predictions {empty}: {predictions!r}")
        sys.exit(1)

    berlin = predictions[0].get("genero")
    print(f"yahoo/Germany/Berlin -> {berlin} ({'Metal, as the previous model' if berlin == 'Metal' else 'not
Metal'})")
    print("smoke test passed")


if __name__ == "__main__":
    main()
```